

Olympus: An Evidence-Bounded Architecture for Specialized Research-Agent Models

Olympus Research Project

Architecture proposal — 6 September 2026

Abstract

Research-agent projects often accumulate branded “models” before they have datasets, checkpoints, interfaces, or falsifiable evaluations. We present a counterproposal for six planned Olympus families: Hermes for grounded response, Prometheus for scientific synthesis, Perseus for tool execution, **Olympus-Atlas** for retrieval over governed corpora, Kronos for temporal planning and guarded adaptation, and Aion for governed research loops. The families share a revision-pinned decoder, typed workspace, immutable evidence graph, and grammar-constrained output envelopes; they differ through adapters, auxiliary heads, services, and promotion criteria. Deterministic systems retain access control, permission checks, transactions, protocol freezing, audit, rollback, and promotion. We specify module boundaries, objectives, data requirements, threat models, matched-budget baselines, ablations, and rejection rules for each family. We also argue that Aion should begin as a deterministic controller, not a monolithic autonomous-scientist model, and that its learned router should be removed unless it beats that controller at equal budget and false-promotion rate. This manuscript is an architecture and evaluation protocol. It reports *no trained family checkpoint and no family-level experimental result*. Its contribution is therefore a falsifiable research plan rather than a claim of model capability.

1 Introduction

Language models can retrieve external knowledge, invoke tools, explore multiple reasoning paths, model trajectories, and participate in automated research loops. These ingredients do not automatically compose into a reliable research agent. Retrieval can return irrelevant or unauthorized evidence; syntactically valid tool calls can be semantically wrong; multiple reasoning traces can agree on the same defect; online adaptation can erase prior capabilities; and a model that proposes a protocol cannot be allowed to approve its own deviations.

The Olympus repository contains real lifecycle infrastructure and small verification models, but its named model families remain specifications. In particular, the planned Hermes base experiment has not produced a qualifying checkpoint, and Prometheus, Perseus, Atlas, Kronos, and Aion have no qualifying datasets or evaluations. We preserve that boundary. The question addressed by this paper is not “how capable are these models?” but “what is the smallest coherent architecture that could test whether the proposed specializations add value?”

Our central decision is that six roles do not justify six foundation-model pretraining programs. We instead propose one shared substrate with independently versioned adapters and services. This yields four contributions:

1. a typed contract separating language proposals from authorized state transitions;
2. concrete architectures for six roles, including family-specific data, losses, interfaces, baselines, failure modes, and stop rules;
3. a dependency and promotion protocol that prevents downstream branding from substituting for upstream evidence; and
4. a paper-level claim ledger that labels every design as proposed and every future numerical claim as inadmissible without immutable run evidence.

The proposal is deliberately scale-skeptical. The first compatible base is the already selected Qwen3-0.6B-Base candidate; Qwen3 reports dense and mixture-of-experts variants starting at the 0.6B scale [Yang et al., 2025]. This citation supports a compatibility decision, not a transfer of reported Qwen3 results to Olympus. A larger base is considered only after data quality and system-level evaluation are demonstrated at the cheapest credible scale.

2 Status and claim boundary

Table 1 is part of the methodology. It prevents architectural intent from being read as completion. “Proposed” means a written, testable design; it does not mean instantiated weights or a functioning service.

Artifact	Status	Evidence allowed in this paper
Shared substrate	PROPOSED	interfaces, losses, promotion rules
Hermes	PROPOSED	base-candidate decision; no family checkpoint
Prometheus	PROPOSED	proposer/verifier architecture only
Perseus	PROPOSED	action protocol and safety boundary only
Olympus-Atlas	PROPOSED	retrieval/index architecture only
Kronos	PROPOSED	offline temporal/adaptation protocol only
Aion	PROPOSED	deterministic state machine and optional router

Table 1: Truth boundary at manuscript freeze. No row contains a family-level experimental result.

We use **Olympus-Atlas** externally because “Atlas” already denotes a retrieval- augmented language model [Izacard et al., 2022]. All future checkpoint IDs must similarly carry an **olympus-** prefix and may use a family name only after promotion. This is both a naming correction and a provenance safeguard.

3 Related architectural evidence

Backbone. The Transformer supplies the common sequence-modeling substrate [Vaswani et al., 2017]. Rotary position embeddings and grouped-query attention provide relative-position and inference-efficiency mechanisms [Su et al., 2021, Ainslie et al., 2023]. Olympus inherits these from the candidate backbone; it does not claim to revalidate them.

Retrieval and memory. Retrieval-augmented generation and RETRO show ways to combine parametric sequence models with external corpora [Lewis et al., 2020, Borgeaud et al., 2021]. ColBERT provides an efficient late-interaction ranking design [Khattab and Zaharia, 2020]. However, long-context models can be position-sensitive, with relevant information in the middle used less reliably [Liu et al., 2023a]. MemGPT demonstrates an operating-system- inspired hierarchy for moving information through limited context [Packer et al., 2023]. These results motivate explicit retrieval, memory tiers, and position-aware tests; they do not make generated citations faithful or memory writes trustworthy by default.

Reasoning, verification, and action. ReAct interleaves reasoning and acting, while Toolformer trains language models to invoke APIs [Yao et al., 2022, Schick et al., 2023]. PICARD shows that incremental parsing can constrain outputs in a formal language [Scholak et al., 2021]. Self-consistency and Tree of Thoughts explore multiple inference paths [Wang et al., 2022, Yao et al., 2023]; process supervision provides a basis for scoring intermediate steps [Lightman et al., 2023]. A separate line of work studies whether models can estimate correctness and knowledge [Kadavath et al., 2022]. We take the conservative intersection: expose checkable states, constrain formal actions, measure calibration, and never equate model confidence with authority.

Temporal learning and agents. Decision Transformer formulates offline decision making as conditional sequence modeling [Chen et al., 2021]; Mamba provides a selective state-space alternative for long sequences [Gu and Dao, 2023]. Elastic weight consolidation is a classic regularization approach to catastrophic forgetting [Kirkpatrick et al., 2016]. Each is an ablation candidate for Kronos, not a preselected winner. AgentBench, τ -bench, and ToolSandbox emphasize interactive and stateful evaluation [Liu et al., 2023b, Yao et al., 2024, Lu et al., 2024b]. Constitutional AI studies supervision from explicit principles [Bai et al., 2022], while The AI Scientist demonstrates the active direction of end-to-end research automation [Lu et al., 2024a]. Olympus differs by keeping authorization, audit, and promotion outside the learned policy.

4 Shared evidence-and-action substrate

4.1 Backbone and specialization

Let B_θ be a revision-pinned decoder-only Transformer and A_f a family-specific adapter. For family f , the learned policy is

$$p_f(y | x) = B_{\theta, A_f}(x), \quad \theta \text{ frozen in the first stage.} \quad (1)$$

Adapters target attention and feed-forward projections. Auxiliary heads may read final hidden states for evidence entailment, confidence, outcome value, or policy-violation prediction. Retrieval encoders and process verifiers may be separate small bidirectional models where independence or latency matters.

The first implementation supplies retrieved evidence as prefix tokens rather than adding cross-attention. This choice minimizes surgery to the base and makes “retrieval quality” separable from “decoder use of retrieved evidence.” A cross-attention variant is eligible only if the prefix baseline saturates.

4.2 Workspace state

Every family reads a serialized `WorkspaceState`:

$$W = (o, c, b, E, C, G, T, P, H, M), \quad (2)$$

where o is the objective, c constraints, b the remaining resource budget, E immutable evidence references, C claims, G a plan DAG, T tool schemas, P permissions and approvals, H observation history, and M exact model and environment identities. Evidence references include content hash, source URI, acquisition time, license, access label, and span offsets. Claims have explicit support, refute, and unknown edges.

The model emits exactly one grammar-constrained envelope:

```
Answer{claims,citations,confidence,abstention?}  
Proposal{hypotheses,tests,assumptions,confidence}  
Action{tool,schema,args,preconditions,idempotency}  
Stop{reason,unresolved,required_authority?}.
```

No family writes directly to external state. A deterministic validator parses the envelope, checks access and permissions, executes a permitted transition, and appends an observation. Untrusted tool text cannot modify policy fields. Confidence is an evaluated prediction, never an authorization mechanism.

4.3 Learning objective and evidence protocol

For a family f , the general objective is

$$\mathcal{L}_f = \mathcal{L}_{LM} + \lambda_s \mathcal{L}_{schema} + \lambda_a \mathcal{L}_{attribution} + \lambda_c \mathcal{L}_{calibration} + \lambda_p \mathcal{L}_{policy} + \mathcal{L}_f^{specific}. \quad (3)$$

Loss weights and stopping rules are frozen before sequestered evaluation. Each component requires an ablation; otherwise the architecture cannot establish which mechanism mattered. Training records must be license-cleared, deduplicated, PII-screened, manifest-bound, and split by source or environment rather than adjacent rows. Teacher outputs carry teacher revision and rights metadata and cannot enter held-out sets.

Results are reported at three levels: component metrics, end-to-end final state, and safety/system integrity. Every experiment retains per-example outputs, dataset and code hashes, checkpoint identity, seeds, latency, peak memory, tokens, tool calls, failures, and confidence intervals. Required baselines are the unadapted base, a deterministic non-LLM system, a single shared adapter, and the proposed specialization under matched budgets.

5 Family architectures

5.1 Hermes: grounded interpretation and response

Hermes is the only user-facing answer generator. Its pipeline is request interpretation $\rightarrow$ authorized retrieval $\rightarrow$ context composition $\rightarrow$ grounded decoding $\rightarrow$ attribution and confidence validation. A request adapter decomposes compound intents and emits retrieval queries. Retrieved passages retain source IDs beside their spans and are packed under a fixed token budget. The decoder produces an `Answer`; an entailment head scores each claim/evidence pair

as support, refute, or insufficient; a confidence head predicts answer correctness and evidence sufficiency. A memory selector can only *propose* a write. A separate ACL and provenance service decides whether it persists.

Hermes training requires licensed question/evidence/answer triples, contradictions, answerable and unanswerable cases, citation spans, memory-consent examples, and prompt-injection negatives. Its additional losses cover claim attribution, abstention, Brier-style confidence, and memory-write selection. Splits are by document and source. Evaluation reports answer correctness, citation precision/recall, entailment, expected calibration error, Brier score, risk-coverage, unsupported-claim rate, memory leakage, latency, and peak memory.

Baselines are no retrieval, BM25 retrieval, and the unadapted base with the same context. The family is rejected if gains vanish after document-level decontamination or if more citation markers do not increase supported claims. Promotion also requires existing Foundry gates for held-out scale, quantization, fresh-process serving, license, safety, and model cards.

5.2 Prometheus: scientific synthesis by proposer and verifier

Prometheus formalizes a question into variables, assumptions, candidate hypotheses, evidence gaps, and falsification tests. A proposer adapter samples a bounded hypothesis lattice. Branches store compact claims and planned checks, not unverifiable private prose. Calculators, code sandboxes, proof checkers, and retrieval attach receipts to branches. A process-verifier adapter scores each checkable transition; an evidence adjudicator preserves conflicts rather than forcing consensus. A selector uses verifier score, evidence coverage, branch diversity, and remaining budget to select or abstain, after which Hermes renders the user-facing result.

Proposer and verifier initially share frozen backbone weights but use disjoint adapters and heads. To measure correlated error, training sources and evaluator authors are split. If correlation remains high, the verifier moves to a different architecture before the proposer scales. Data include licensed proofs, executable mathematics, systematic reviews with source spans, contradictory and retracted claims, negative results, and perturbed invalid steps. Specific objectives measure branch diversity, process validity, contradiction recognition, and final outcome.

Matched-compute baselines are greedy decoding, self-consistency, and tree search without a learned verifier. Outcomes include exact answers, verified-step precision/recall, citation entailment, contradiction detection, calibration, abstention, cost, and proposer/verifier error correlation. Reject Prometheus if equal-compute sampling matches it or if verifier selection favors persuasive invalid branches over deterministic outcome checks.

5.3 Perseus: typed tool action and recovery

Perseus serializes goal, observations, budget, tool schemas, and transaction status, then emits an **Action** through incremental grammar-constrained decoding. A non-learned capability kernel resolves the exact tool and schema, validates argument types and preconditions, checks permissions and approval, assigns an idempotency key, and executes in a sandbox. An observation normalizer records structured outputs and partial effects while isolating untrusted tool content. A recovery critic classifies failures as retryable, compensatable, fatal, or authority-required. Material writes require a checkpoint and explicit commit; retries reuse their idempotency key.

Training traces must contain successes, failures, minimal-tool solutions, permission denials, stale observations, partial writes, compensating actions, and tool-output prompt injection. Losses cover tool selection, typed arguments, preconditions, outcome prediction, recovery, and unsafe attempts. Evaluation uses resettable stateful environments and compares deterministic workflows, ReAct prompting, unconstrained function calling, and Perseus without recovery. Primary metrics are final-state equality, schema validity, tool/argument accuracy, excess calls, recovery, idempotency, budget, false refusal, unsafe attempts, and actual side effects. A textual statement of success has zero weight. Any unauthorized write blocks promotion.

5.4 Olympus-Atlas: governed retrieval and attribution

Olympus-Atlas ingests documents into overlapping semantic passages while preserving document/version hashes, timestamps, license, ACL, and original span offsets. Access filtering occurs *before* lexical or neural scoring so ranks do not expose hidden corpus membership. Candidate generation fuses BM25 and a compact bi-encoder. A ColBERT-style late-interaction model reranks only authorized candidates. A context composer selects diverse passages under a token budget, keeps source IDs inline, and varies evidence position. An attribution adapter generates evidence graphs; a separate entailment head validates edges. Immutable indexes swap atomically and remain replayable by version.

Training uses query/passage judgments, within-document hard negatives, temporal replacements, contradictions, ACL canaries, and multi-document questions. It optimizes contrastive retrieval, late-interaction ranking, freshness

selection, attribution, and abstention. Metrics include Recall@ k , nDCG@10, answer and citation correctness, entailment, freshness, evidence-position sensitivity, index lag, latency, memory, and ACL leakage. Baselines are BM25, dense-only, long-context without retrieval, hybrid without reranking, and the full system. Zero ACL leakage is a hard gate. Do not extend context if retrieval remains the binding error, and reject the neural stack if hybrid search cannot beat BM25 at matched latency.

5.5 Kronos: temporal planning and guarded adaptation

Kronos converts timestamped observations, interventions, delayed outcomes, missingness, and environment versions into event tokens. A compact selective state-space encoder summarizes long streams; a Transformer readout can attend to retrieved raw episodes. The state-space module is explicitly an ablation. A forecaster emits calibrated outcome distributions at declared horizons. A return- and constraint-conditioned planner emits a receding-horizon plan DAG with checkpoints and stop conditions. After Perseus executes an authorized step, an observer compares predicted and actual state and triggers replanning.

Adaptation occurs offline. A manager trains a new adapter on a deletion-aware replay set and compares no update, naive fine-tuning, replay, and regularization. Only the promotion service can atomically swap adapters. Every plan and adapter records environment, data, base, and predecessor hashes; rollback is tested before promotion.

Data include timestamped trajectories, interventions, delayed or censored outcomes, drift episodes, and clearly labeled simulations. Adjacent time points cannot cross train/test boundaries. Forward-chaining evaluation reports calibration by horizon, plan success, regret, recovery, adaptation gain, backward transfer/forgetting, deletion compliance, and rollback identity. Online weight mutation is forbidden until a predeclared forgetting bound and a fresh-process rollback test pass.

5.6 Aion: governed research-loop controller

Aion is primarily a protocol state machine:

$$\text{question} \rightarrow \text{evidence} \rightarrow \text{hypotheses} \rightarrow \text{frozen protocol} \rightarrow \text{authorized execution} \rightarrow \text{audit} \rightarrow \{\text{revise, stop, review}\}. \quad (4)$$

Every transition has typed inputs, a deterministic admission predicate, a budget, an authorized actor, and an immutable receipt. Inspecting outcomes can never silently modify a frozen protocol; any change creates a post-hoc version.

A deterministic controller and routing table are the required baseline. Prometheus proposes hypotheses and protocols; **Olympus-Atlas** maps claims to sources; Perseus performs bounded execution; Kronos monitors delayed outcomes; and Pantheon audits canonical packages and evidence identities. Protected test answers are invisible to proposers. Learned components cannot waive audit or promotion. A learned router may predict the next permitted service and budget, but deterministic masks make illegal transitions unreachable. External writes, publication, high-cost jobs, and evidence promotion require human authority.

The optional router trains only on versioned traces containing negative results, interruptions, permission denials, and independently reviewed claim maps. Evaluation measures valid-experiment rate, protocol deviation, false-promotion rate, claim precision, evidence coverage, protected-data leakage, budget, intervention recovery, and external reproduction. The decisive falsification criterion is simple: remove the learned Aion component if a deterministic controller with identical tools and budget matches valid-experiment rate at equal or lower false-promotion rate. Self-approval, protocol rewriting after outcomes, hidden-test access, unbounded recursion, or an unauthorized effect is a hard failure.

6 End-to-end dependency and deployment plan

The dependency order is Foundry $\rightarrow$ Hermes $\rightarrow$ {Prometheus, Perseus} $\rightarrow$ **Olympus-Atlas** $\rightarrow$ Kronos $\rightarrow$ Aion, with Pantheon auditing every stage. This is a promotion dependency, not a ban on prototyping interfaces in parallel. Aion can exist immediately as a deterministic state machine, but no learned autonomous behavior can be promoted before the relevant upstream services and rollback controls are real.

Serving loads one quantized backbone and switches signed adapter bundles. Retrieval, capability, transaction, temporal, and audit services are separate processes with narrow schemas. This separation costs some latency but preserves ownership: a neural decoder cannot become the de facto security boundary. Each request records base and adapter hashes, service versions, corpus version, policy version, token/tool budget, and every state transition.

Family	Primary learned object	Hard external boundary	First decisive comparison
Hermes Prometheus Perseus	grounded adapter + heads proposer/verifier adapters action/recovery adapters	ACL memory and citation validation executable checks and evidence graph capability kernel and transaction manager	adapted vs unadapted base verifier vs equal-compute sampling constrained vs unconstrained action
Olympus-Atlas Kronos Aion	retriever/reranker + attribution temporal encoder + planner optional router	ACL filter and immutable index offline update gate and rollback protocol, audit, approval, promotion	hybrid vs BM25 at matched latency temporal hybrid vs static replanning learned vs deterministic controller

Table 2: Minimal architecture and falsification target for each family.

7 Experimental program

7.1 Phase 0: contracts and deterministic baselines

Implement schema definitions, event logging, validators, ACL filtering, capability checks, transactions, rollback, and the Aion state machine before training. Construct deterministic BM25, workflow, static replanning, and routing baselines. Freeze test generators and failure injections. Exit only when a fresh process can replay a run from hashes and rejects malformed, unauthorized, or post-hoc transitions.

7.2 Phase 1: Hermes at 0.6B

Pin and hash the selected base revision and license. Run supervised adapter training at the repository’s proposed bounded settings, but treat them as hyperparameters rather than results. Compare base-only, prompting, and Hermes on document-held-out grounding and out-of-domain calibration. Exit only with a reproducible multi-seed advantage on the primary endpoint and no safety-gate regression. A failed result is publishable evidence against specialization and does not authorize a larger base.

7.3 Phase 2: orthogonal capability tests

Train Prometheus proposer/verifier and Perseus action/recovery adapters independently. For Prometheus, freeze an equal-inference-token comparison. For Perseus, reset environments per episode and score final state. Do not combine them until both pass component and safety gates; otherwise integration obscures root causes.

7.4 Phase 3: knowledge and time

Build **Olympus-Atlas** as a retrieval experiment before any context extension. Freeze corpus versions and ACL canaries. Only after a reliable evidence service exists should Kronos compare Transformer-only, state-space hybrid, and static replanning. Continual adaptation stays offline until deletion, retention, and rollback tests pass.

7.5 Phase 4: governed loop

Run Aion’s deterministic controller across bounded research tasks. Then add the learned router in a paired, identical-budget evaluation. Pantheon audits both. The learned variant is accepted only if its valid-experiment improvement is accompanied by a predeclared non-inferiority bound on false promotion and zero protected-data or authorization violations.

For all phases, use at least three seeds, bootstrap intervals over the true unit of sampling, and report all preregistered outcomes. Family-level claims require an independently executed reproduction after the development test is frozen.

8 Threat model and reliability properties

The system assumes the language model, retrieved text, tool output, and training examples can all be adversarial. Threats include prompt injection, corpus membership leakage, stale evidence, fabricated citation spans, schema-valid but dangerous arguments, duplicate side effects, reward hacking, protocol drift, held-out-answer access, checkpoint substitution, and self-promotion.

The principal controls are pre-retrieval ACL filtering, content-addressed evidence, constrained decoding, deterministic semantic validation, least- privilege tools, sandboxing, idempotency keys, two-phase material writes,

immutable logs, versioned protocols, separated proposer/verifier data, sequestered tests, offline updates, atomic rollback, and human approval. These controls do not prove security. They create inspectable failure points and prevent the learned model from being the sole enforcement layer.

Reliability is defined over final states and receipts. An answer can be fluent and wrong; an action can be valid JSON and unsafe; a run can succeed once and fail under reset. Consequently, promotion requires replay from a fresh process, environment reset, dependency and model identity checks, and retained failure artifacts. Silent coercion of missing outputs to success is forbidden.

9 Limitations and negative hypotheses

The modular design may impose latency and coordination overhead. Shared weights may create correlated errors between families; separate adapters do not ensure independence. Small bases may lack the capacity needed for scientific synthesis or robust tool recovery. Confidence heads can be miscalibrated out of domain. Retrieval pipelines can amplify source bias. State-space encoders may not help the sparse, irregular event streams available to Kronos. Deterministic Aion may outperform a learned router precisely because its state space is well specified.

These are not implementation inconveniences; they are negative hypotheses the program must be prepared to accept. The project should merge or delete a family whose specialization does not beat the simplest valid baseline. It should not rescue a weak claim by scaling parameters, changing metrics after evaluation, or collapsing multiple roles into an uninspectable agent.

10 Conclusion

Olympus is most defensible as an evidence-bounded architecture program, not a collection of prematurely named models. One shared decoder supports economical specialization; explicit services preserve retrieval, action, time, and audit boundaries; deterministic systems keep authority outside learned policies. Each family has a primary hypothesis and an explicit rejection rule. The immediate research direction is therefore contracts and baselines, followed by one Hermes 0.6B adapter experiment. Prometheus and Perseus come next, **Olympus-Atlas** and Kronos only after their prerequisites, and Aion remains a governed controller until learning demonstrates value. The proposal becomes a model paper only after those experiments exist. Until then, its scientifically correct output is the protocol itself.

Artifact statement

The repository accompanies this manuscript with one shared-substrate specification, six family architecture files, a claim-source ledger, a canonical research report, BibTeX references, and this source. No generated number in a later revision is valid unless linked to an immutable run manifest.

References

- Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints, 2023. URL <https://arxiv.org/abs/2305.13245>.
- Yuntao Bai et al. Constitutional ai: Harmlessness from ai feedback, 2022. URL <https://arxiv.org/abs/2212.08073>.
- Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, et al. Improving language models by retrieving from trillions of tokens (RETRO), 2021. URL <https://arxiv.org/abs/2112.04426>.
- Lili Chen, Kevin Lu, Aravind Rajeswaran, et al. Decision transformer: Reinforcement learning via sequence modeling, 2021. URL <https://arxiv.org/abs/2106.01345>.
- Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2023. URL <https://arxiv.org/abs/2312.00752>.

- Gautier Izacard, Patrick Lewis, Maria Lomeli, et al. Atlas: Few-shot learning with retrieval augmented language models, 2022. URL <https://arxiv.org/abs/2208.03299>.
- Saurav Kadavath et al. Language models (mostly) know what they know, 2022. URL <https://arxiv.org/abs/2207.05221>.
- Omar Khattab and Matei Zaharia. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT, 2020. URL <https://arxiv.org/abs/2004.12832>.
- James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, et al. Overcoming catastrophic forgetting in neural networks, 2016. URL <https://arxiv.org/abs/1612.00796>.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks, 2020. URL <https://arxiv.org/abs/2005.11401>.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, et al. Let’s verify step by step, 2023. URL <https://arxiv.org/abs/2305.20050>.
- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranajpe, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts, 2023a. URL <https://arxiv.org/abs/2307.03172>.
- Xiao Liu, Hao Yu, Hanchen Zhang, et al. AgentBench: Evaluating LLMs as agents, 2023b. URL <https://arxiv.org/abs/2308.03688>.
- Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI Scientist: Towards fully automated open-ended scientific discovery, 2024a. URL <https://arxiv.org/abs/2408.06292>.
- Jiarui Lu, Thomas Holleis, Yizhe Zhang, et al. ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities, 2024b. URL <https://arxiv.org/abs/2408.04682>.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems, 2023. URL <https://arxiv.org/abs/2310.08560>.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, et al. Toolformer: Language models can teach themselves to use tools, 2023. URL <https://arxiv.org/abs/2302.04761>.
- Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. PICARD: Parsing incrementally for constrained autoregressive decoding from language models, 2021. URL <https://arxiv.org/abs/2109.05093>.
- Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding, 2021. URL <https://arxiv.org/abs/2104.09864>.
- Ashish Vaswani et al. Attention is all you need, 2017. URL <https://arxiv.org/abs/1706.03762>.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, et al. Self-consistency improves chain of thought reasoning in language models, 2022. URL <https://arxiv.org/abs/2203.11171>.
- An Yang et al. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models, 2022. URL <https://arxiv.org/abs/2210.03629>.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, et al. Tree of Thoughts: Deliberate problem solving with large language models, 2023. URL <https://arxiv.org/abs/2305.10601>.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. tau-bench: A benchmark for tool-agent-user interaction in real-world domains, 2024. URL <https://arxiv.org/abs/2406.12045>.